Análisis de Tamaño de Datos y Comunicación — Crozzo POS

> Fecha: 2026-11-08 · Objetivo: analizar cuánto pesa cada mensaje que se envía entre dispositivos, qué se comunica, y si Bluetooth/malla es viable.

---

1. Tamaño real de los datos que se transmiten

1.1 Comanda (pedido a cocina)

Estructura de una comanda (de `crozzoCreateComandaFromTransfer`):

```json
{
  "id": 123,
  "transaction_id": "uuid-v4",
  "origen": "caja",
  "tipoServicio": "mesa",
  "referencia": "Mesa 5",
  "areaId": "cocina",
  "areaNombre": "Cocina",
  "envioNum": 1,
  "estado": "pendiente",
  "createdAt": "2026-11-08T10:30:00.000Z",
  "lastUpdateAt": "2026-11-08T10:30:00.000Z",
  "items": [
    { "id": "prod-1", "nombre": "Hamburguesa", "cantidad": 2, "precio": 15000, "nota": "" },
    { "id": "prod-2", "nombre": "Coca-Cola", "cantidad": 1, "precio": 5000, "nota": "sin hielo" }
  ],
  "total": 35000,
  "impresora": "POS-1",
  "estilo": { "fontSize": 12, "showPrice": false, "showHeader": true },
  "sender": { "deviceId": "tablet-1", "role": "mesero" }
}
```

Tamaño estimado:
Componente	Bytes
Header (id, tipo, referencia, estado, fechas)	~200 bytes
Por cada item (id, nombre, cantidad, precio, nota)	~80-120 bytes
Impresora + estilo + sender	~100 bytes
Comanda de 3 items	**~500-700 bytes**
Comanda de 10 items	**~1,200-1,500 bytes**

1.2 Runtime sync (estado de mesas)

El runtime snapshot (`crozzo_mesa_runtime`) contiene:
- Estado de todas las mesas (ocupada/libre, carrito, cliente)
- Runtime de slots (mesa/llevar)

Tamaño estimado:
Componente	Bytes
Por mesa (id, estado, carrito resumido)	~100-200 bytes
20 mesas activas	~2,000-4,000 bytes
Runtime completo (todas las mesas + slots)	~5,000-10,000 bytes

1.3 Factura (venta completa)

```json
{
  "uuid": "uuid-v4",
  "consecutivo": 123,
  "fechaEmision": "2026-11-08T10:30:00.000Z",
  "tipoDocumento": "01",
  "compradorNit": "900123456-7",
  "compradorNombre": "Cliente",
  "items": [{ "descripcion": "...", "cantidad": 2, "precio": 15000, "costoUnitario": 8000 }],
  "total": 35000,
  "inventarioMeta": { "version": 1, "aplicado": true, "lineas": [...] },
  "costoMercanciaVendida": 16000,
  "utilidadBruta": 19000
}
```

Tamaño estimado:
Componente	Bytes
Header (uuid, fecha, cliente)	~200 bytes
Por cada item (descripción, cantidad, precio, costo)	~100-150 bytes
inventarioMeta (auditoría por línea)	~200-400 bytes
CMV + utilidad	~50 bytes
Factura de 5 items	**~1,500-2,500 bytes**

1.4 Digest / Presence (muy pequeño)

```json
{ "type": "presence", "deviceId": "tablet-1", "role": "mesero", "status": "online", "ts":
1731050000 }
```

Tamaño: ~80-120 bytes

1.5 Resumen de tamaños

Mensaje	Tamaño típico	Máximo
Comanda (3-5 items)	500-800 bytes	1,500 bytes
Comanda (10+ items)	1,200-1,500 bytes	2,000 bytes
Estado de mesas (20 mesas)	2,000-4,000 bytes	8,000 bytes
Factura (5 items)	1,500-2,500 bytes	4,000 bytes
Presence/digest	80-120 bytes	200 bytes
Sync runtime completo	5,000-10,000 bytes	20,000 bytes

---

2. Frecuencia de comunicación

2.1 Escenario típico (restaurante con 3 dispositivos)

Evento	Frecuencia	Tamaño	Dispositivos
Tomar pedido (comanda)	1-5/min	500-1,500 bytes	Tablet → Caja → Cocina
Cambio de estado (preparando/listo)	1-5/min	100-200 bytes	Cocina → Caja/Tablet
Cobro	1-3/min	1,500-2,500 bytes	Caja → Nube/Tablet
Sync runtime (mesas)	Cada 1-4s	2,000-4,000 bytes	Bidireccional
Presence (heartbeat)	Cada 5-10s	80-120 bytes	Bidireccional
Factura DIAN	1-3/min	3,000-8,000 bytes (XML)	Caja → DIAN

2.2 Volumen total por hora (estimado)

Flujo	Mensajes/hora	Bytes/hora	KB/hora
Comandas	60-300	30,000-450,000	30-450 KB
Cambios de estado	60-300	6,000-60,000	6-60 KB
Cobros	20-60	30,000-150,000	30-150 KB
Sync runtime	900-3,600	1,800,000-14,400,000	1.8-14.4 MB
Presence	360-720	28,800-86,400	29-86 KB
Facturas DIAN	20-60	60,000-480,000	60-480 KB
Total	**~1,420-4,980**	**~1.9-15.6 MB**	**~1.9-15.6 MB/hora**

2.3 Comparativa con capacidades

Canal	Ancho de banda	Latencia	Viabilidad
WiFi LAN (802.11n)	50-150 Mbps	1-5 ms	✅ Sobrado
WiFi LAN (802.11ac)	200-800 Mbps	1-3 ms	✅ Sobrado
Bluetooth LE	1-2 Mbps	10-30 ms	⚠️ Solo comandas/presence
Bluetooth Classic	2-3 Mbps	10-20 ms	⚠️ Solo datos pequeños
Internet (4G)	5-50 Mbps	30-100 ms	✅ Sobrado
Internet (3G)	0.5-5 Mbps	100-300 ms	⚠️ OK para sync, lento para runtime
Internet (Edge)	0.1-0.3 Mbps	300-800 ms	❌ Insuficiente para runtime sync

---

3. Análisis por dispositivo

3.1 Tablet (Rol B — mesero)

Qué envía:
- Comandas (500-1,500 bytes) cada 1-5 min
- Cambios de estado (100-200 bytes) cada 1-5 min
- Presence (80-120 bytes) cada 5-10s
- Sync runtime pull (2,000-4,000 bytes) cada 1-4s

Qué recibe:
- Actualizaciones de mesas (2,000-4,000 bytes) cada 1-4s
- Cambios de estado de comandas (100-200 bytes)
- Notificaciones (impresión, etc.)

Volumen total por tablet: ~2-5 MB/hora (principalmente sync runtime)

Capacidad necesaria:
- WiFi: ✅ Sobrado (50+ Mbps)
- Bluetooth LE: ⚠️ Límite (1-2 Mbps = 125-250 KB/s, pero sync runtime cada 1-4s de 2-4KB es OK)
- 4G: ✅ Sobrado (5-50 Mbps)
- 3G: ⚠️ Justo (0.5-5 Mbps, sync runtime cada 4s = 1KB/s = 8 Kbps, OK)

3.2 Caja central (Rol A)

Qué envía:
- Sync runtime (5,000-10,000 bytes) cada 1-4s a todas las tablets
- Cambios de estado de comandas (100-200 bytes)
- Facturas DIAN (3,000-8,000 bytes) cada 1-3 min
- Presencia (80-120 bytes) cada 5-10s
- Notificaciones a cocina (impresión)

Qué recibe:
- Comandas de todas las tablets (500-1,500 bytes cada)
- Cambios de estado (100-200 bytes)
- Presencia de todas las tablets

- Sync runtime de tablets (2,000-4,000 bytes cada)

Volumen total por caja: ~5-20 MB/hora (depende de número de tablets)

Capacidad necesaria:
- WiFi: ✅ Sobrado
- Bluetooth LE: ❌ Insuficiente (muchos dispositivos, alto volumen)
- 4G: ✅ OK
- Disco local: ✅ (SQLite + localStorage, 4.5MB tope en localStorage)

3.3 Cocina (Rol cocina)

Qué recibe:
- Comandas nuevas (500-1,500 bytes) cada 1-5 min
- Cambios de estado (100-200 bytes)

Qué envía:
- Cambios de estado (preparando/listo/entregada) (100-200 bytes)
- Presencia (80-120 bytes)

Volumen total: ~1-3 MB/hora

---

4. ¿Bluetooth es viable?

4.1 Bluetooth LE (BLE)

Característica	Valor	Suficiente para POS?
Ancho de banda	1-2 Mbps (125-250 KB/s)	⚠️ Para comandas sí, para sync runtime ajustado
Latencia	10-30 ms	✅ OK
Alcance	10-30 metros	⚠️ Solo misma sala
Conexiones simultáneas	1-7 (depende del OS)	❌ Insuficiente para 3+ tablets
Consumo de batería	Bajo	✅ OK

Conclusión BLE: Viable **solo** para:
- Comandas pequeñas (500-1,500 bytes)
- Cambios de estado (100-200 bytes)
- Presencia (80-120 bytes)
- **NO** para sync runtime (2,000-10,000 bytes cada 1-4s)
- **NO** para facturas DIAN (3,000-8,000 bytes)
- **NO** para más de 2-3 dispositivos simultáneos

4.2 Bluetooth Classic

Característica	Valor	Suficiente para POS?
Ancho de banda	2-3 Mbps (250-375 KB/s)	⚠️ Mejor que BLE pero sigue limitado
Latencia	10-20 ms	✅ OK
Alcance	10-100 metros	✅ OK
Conexiones simultáneas	7 (piconet)	⚠️ Límite
Consumo de batería	Alto	❌ No ideal para tablets

Conclusión Classic: Mejor que BLE pero sigue siendo limitado. No recomendado para POS completo.

4.3 Veredicto: ¿Bluetooth para Crozzo POS?

NO para sync completo. Bluetooth no puede manejar:
- Sync runtime (2,000-10,000 bytes cada 1-4s × N dispositivos)
- Facturas DIAN (3,000-8,000 bytes)

- Más de 3-4 dispositivos simultáneos

SÍ para casos específicos:
- Impresoras Bluetooth (tickets térmicos)
- Dispositivos cercanos sin WiFi (backup de emergencia)
- Transferencia de comandas pequeñas (fallback de último recurso)

---

5. ¿Qué se puede cifrar y cómo?

5.1 Lo que ya se cifra
Dato	Cifrado	Método
QR de pareo	✅	ECDH P-256 + firma
Token LAN	✅	Generado criptográficamente
Sync HTTP	✅	TLS (HTTPS/WSS)
Cloud (Supabase)	✅	TLS

5.2 Lo que NO se cifra (y debería)
Dato	Estado	Riesgo
Comandas en LAN (HTTP)	❌ Sin cifrado adicional	Interceptable en red local
Comandas en LAN (WS)	❌ Sin cifrado adicional	Interceptable en red local
Runtime sync (mesas)	❌ Sin cifrado adicional	Interceptable en red local

5.3 Cifrado recomendado para LAN
- **TLS en HTTP/WS**: Usar certificados self-signed o mTLS
- **Payload encryption**: AES-256-GCM para datos sensibles (comandas con info de clientes)
- **Firma de mensajes**: HMAC-SHA256 con token compartido

---

6. Persistencia offline: ¿se pierde información?

6.1 Lo que ya se guarda localmente
Dato	Almacenamiento	Límite
Config (empresa, DIAN, etc.)	localStorage	5-10 MB
Runtime (mesas, slots)	localStorage (`crozzo_pos_runtime_v1`)	4.5 MB tope
Comandas	localStorage (`comandas`)	Sin límite explícito
Operaciones pendientes	SQLite (Tauri) / localStorage	Sin límite
Facturas	localStorage + IndexedDB (posible)	Sin límite explícito

6.2 Qué pasa si se pierde internet
```
1. App detecta offline (navigator.onLine / WS disconnect)
2. Cambia a modo offline
3. Todas las operaciones se guardan localmente
4. Contingencia fiscal se activa (48h SLA para timbrado)
5. Cuando vuelve internet:
   a. Drena outbox (operaciones pendientes)
   b. Sync runtime (push/pull)
   c. Reintenta facturas pendientes
   d. Notifica al usuario: "Sincronizado"
```

6.3 Cómo sabe qué mandó y qué no
```
Cada operación tiene:
  - id único (UUID)
```

```
  - timestamp (createdAt/lastUpdateAt)
  - estado (pendiente/enviado/confirmado)
  - ACK del receptor

Flujo:
  1. Operación creada → estado "pendiente" → guardada local
  2. Enviada → estado "enviado" → espera ACK
  3. ACK recibido → estado "confirmado" → no se reenvía
  4. Sin ACK en X tiempo → reintento automático
  5. Duplicado detectado (por id) → se ignora
```

Mecanismo: Outbox pattern + ACK + idempotency key (consecutivo).

---

7. Realidad vs. expectativa

Escenario	Realidad	Viabilidad
Tablet toma pedido → caja recibe	✅ Funciona (HTTP/WS LAN o cloud)	Alta
Caja → cocina (comanda)	✅ Funciona (WS push + auto-print)	Alta
Caja maneja grandes datos	✅ Funciona (SQLite + sync)	Alta
Tablet sin internet → caja sin internet	✅ Funciona (modo offline + sync después)	Alta
Bluetooth para comandas	⚠️ Solo datos pequeños, 2-3 dispositivos max	Media
Bluetooth para sync completo	❌ Insuficiente (ancho de banda + conexiones)	Baja
Malla Bluetooth (mesh)	❌ No implementado, complejidad alta	Baja
Cifrado en LAN	⚠️ TLS disponible pero no obligatorio	Media
Pérdida de datos	❌ No se pierde (persistencia local + sync)	Alta
Auto-descubrimiento	✅ mDNS implementado	Alta
Fallback automático	✅ LAN → Cloud → Offline	Alta

---

8. Recomendaciones finales

Para comunicación entre dispositivos
1. **Usar mDNS + HTTP/WS** (ya implementado) para misma red
2. **Usar cloud (Supabase)** como fallback para cross-network
3. **NO usar Bluetooth** para sync de datos (solo impresoras)
4. **Implementar WebRTC** como complemento para baja latencia (futuro)

Para seguridad
1. **Obligar TLS** en LAN (certificados self-signed)
2. **Cifrar payloads** con AES-256-GCM para datos sensibles
3. **Firmar mensajes** con HMAC-SHA256

Para persistencia offline
1. **Verificar** que el outbox pattern esté implementado correctamente
2. **Agregar idempotency key** a todas las operaciones
3. **Monitorear** el tamaño de localStorage (4.5MB tope)

Para escalabilidad
1. **Caja central** debe usar SQLite (no localStorage) para grandes volúmenes
2. **Tablets** pueden usar localStorage (datos pequeños)
3. **Sync runtime** debe ser incremental (solo cambios, no estado completo)

---

9. Conclusión

- **Los mensajes son pequeños** (500-1,500 bytes por comanda, 2,000-4,000 bytes por sync runtime).
- **Bluetooth NO es viable** para sync completo (ancho de banda + conexiones simultáneas).
- **mDNS + HTTP/WS + Cloud** es la estrategia correcta y ya está implementada.
- **No se pierde información** (persistencia local + outbox + sync).
- **El cifrado existe** en cloud y QR, pero falta en LAN (HTTP/WS sin TLS).
- **La auto-descubrimiento funciona** por mDNS (misma red) y cloud (internet).

El sistema está bien diseñado para el volumen de datos que maneja. El principal riesgo es la **falta de cifrado en LAN** y los **bugs latentes** (digest ciego, WS panic).